

KONSPEKT PROJEKTU BADAWCZEGO

Zastosowanie MCTS/UCT do gry Breakthrough 8×8

Metody Sztucznej Inteligencji 2

Hubert Sobociński

Numer indeksu: 327315

Sebastian Rydz

Numer indeksu: 327251

Warszawa, 4 maja 2026

Spis treści

1	Opis problemu	2
1.1	Gra Breakthrough	2
1.2	Dlaczego gra Breakthrough 8×8	2
1.3	Cel projektu	2
2	Wstępny przegląd literatury	3
2.1	Monte Carlo Tree Search i algorytm UCT	3
2.2	RAVE i statystyki AMAF	4
2.3	Progressive Bias	4
2.4	Heurystyka specjalistyczna dla Breakthrough	5
3	Hipotezy badawcze	9
4	Sposób weryfikacji hipotez	10
4.1	Projekt eksperymentów	10
4.2	Dane eksperymentalne	10
4.3	Liczność prób	10
4.4	Metodologia statystyczna	11
4.5	Metryki dodatkowe	11
4.6	Analiza wyników	11
4.7	Reprodukowalność	11
4.8	Testy z udziałem użytkowników	11
5	Harmonogram projektu	12
6	Ogólny projekt techniczny	12
6.1	Stos technologiczny	12
6.2	Zrównoleglenie	12
6.3	Struktura repozytorium	12

1. Opis problemu

1.1. Gra Breakthrough

Breakthrough to dwuosobowa gra strategiczna z pełną informacją, wynaleziona przez Dana Troyka w 2000 roku (zwycięzca konkursu 8×8 Game Design Competition). Plansza jest kwadratowa, taka jak w szachach; każdy gracz dysponuje pionkami ustawionymi w dwóch pierwszych rzędach po swojej stronie. Ruch pionka odbywa się o jedno pole do przodu — prosto lub po skosie. Ruch prosty jest możliwy wyłącznie na puste pole; ruch po skosie może być wykonany na pole puste lub zajęte przez pionek przeciwnika (bicie). Bicie prosto jest niedozwolone, a pionki nie mogą się cofać.

Gra kończy się zwycięstwem gracza, który:

1. doprowadzi pionek do ostatniego rzędu przeciwnika,
2. zbije wszystkie pionki przeciwnika,
3. uniemożliwi przeciwnikowi wykonanie jakiegokolwiek legalnego ruchu.

Remis nie jest możliwy.

1.2. Dlaczego gra Breakthrough 8×8

Gra spełnia wszystkie wymagania projektu:

- **Dwuosobowa, z pełną informacją, deterministyczna** — kluczowe właściwości dla metod MCTS/UCT.
- **Umiarkowany branching factor** — w typowej pozycji na planszy 8×8 liczba legalnych ruchów wynosi 15–30. Głębokość partii to 40–80 ruchów.
- **Proste zasady, ale trudna strategicznie** — mimo prostych zasad wymaga planowania na wiele ruchów naprzód (tworzenie łańcuchów pionków, kontrola kolumn, poświęcenia pionka w celu wygranej).
- **Niszowa** — w odróżnieniu od szachów czy warcabów gra nie jest aż tak popularna.
- **Wariant 8×8 nie został rozwiązany** — Saffidine *i in.* [1] rozwiązyli warianty do planszy 6×5 , a Björnsson *i in.* [2] rozwiązyli planszę 6×6 (w obu przypadkach wygrywa pierwszy gracz). Klasyczna plansza 8×8 pozostaje otwarta, co zostawia miejsce na eksperymenty porównawcze bez znanego „perfekcyjnego gracza”.

1.3. Cel projektu

Celem projektu jest zaprojektowanie i implementacja agentów grających w Breakthrough 8×8 z wykorzystaniem algorytmu Monte Carlo Tree Search w wariacie UCT oraz jego rozszerzeń, a następnie eksperymentalne porównanie ich skuteczności.

W szczególności projekt obejmuje:

- implementację bazowego algorytmu UCT,

- implementację dwóch rozszerzeń UCT:
 - RAVE (z literatury),
 - Progressive Bias (z literatury),
- implementację gracza heurystycznego (minimax alfa-beta),
- porównanie jakości gry powyższych podejść,
- stworzenie interfejsu człowiek–komputer i przeprowadzenie testów z użytkownikami.

2. Wstępny przegląd literatury

2.1. Monte Carlo Tree Search i algorytm UCT

Algorytm Monte Carlo Tree Search (MCTS), spopularyzowany przez Couloma [3] w kontekście gry Go, to asymetryczna metoda przeszukiwania drzewa decyzyjnego, która w wariancie podstawowym nie wymaga wbudowanej, eksperckiej funkcji oceny heurystycznej. Działanie algorytmu opiera się na iteracyjnym powtarzaniu czterech faz:

1. **Selekcja:** Algorytm przechodzi od korzenia w dół drzewa, wybierając węzły na podstawie tzw. polityki drzewa (tree policy), aż do osiągnięcia liścia lub węzła, który nie został w pełni rozwinięty (istnieją niezbadane legalne ruchy).
2. **Ekspansja:** Do drzewa dodawany jest nowy węzeł potomny, odpowiadający jednemu z możliwych ruchów z wybranego stanu.
3. **Symulacja:** Z nowo dodanego węzła rozgrywana jest losowa partia (z użyciem tzw. default policy) aż do osiągnięcia stanu terminalnego gry.
4. **Propagacja wsteczna:** Wynik symulacji (np. wygrana, przegrana) jest propagowany w górę drzewa, aktualizując statystyki odwiedzin i zwycięstw każdego węzła na ścieżce.

Kocsis i Szepesvári [4] zaproponowali algorytm UCT (Upper Confidence Bounds applied to Trees), który optymalizuje fazę selekcji, wykorzystując regułę UCB1, aby zachować balans pomiędzy eksploracją (badaniem nowych ruchów) a eksploatacją (wybieraniem ruchów znanych jako dobre). Wartość wyboru węzła i obliczana jest ze wzoru:

$$UCT_i = \frac{w_i}{n_i} + c \sqrt{\frac{\ln N}{n_i}}$$

gdzie w_i to liczba zwycięstw, n_i to liczba odwiedzin węzła potomnego, N to liczba odwiedzin węzła rodzica, a c to stała eksploracji.

Wykorzystanie w projekcie: Zaimplementujemy klasyczny algorytm UCT, który posłuży jako punkt odniesienia (baseline). Pozwoli on sprawdzić, jak czyste przeszukiwanie losowe radzi sobie w Breakthrough oraz stanowić będzie podstawę do oceny zysków z zaimplementowanych rozszerzeń.

2.2. RAVE i statystyki AMAF

RAVE (Rapid Action Value Estimation), technika wprowadzona przez Gelly’ego i Silvera [5], służy do radykalnego przyspieszenia zbieżności algorytmu poprzez uogólnianie i współdzielenie informacji o jakości ruchów, co opiera się na heurystyce AMAF (All-Moves-As-First).

Zamiast aktualizować statystyki danego ruchu tylko w konkretnym węźle drzewa, AMAF zakłada, że jeśli ruch był skuteczny w dowolnym momencie rozegranej symulacji (playoutu), to prawdopodobnie jest to ruch wartościowy również z poziomu węzła początkowego. RAVE modyfikuje politykę drzewa, łącząc klasyczne oszacowanie MCTS z oszacowaniem AMAF poprzez średnią ważoną:

$$V = (1 - \beta) \cdot V_{UCT} + \beta \cdot V_{AMAF}$$

Waga β jest dynamicznie dobierana i zależy od liczby odwiedzin. Przy małej liczbie symulacji β jest bliskie 1 (dominuje AMAF, co daje szybkie, choć zgrubne rozeznanie), a w miarę wzrostu statystyk β dąży do 0 (dominuje dokładna ocena z UCT).

Wykorzystanie w projekcie: W grze Breakthrough ruchy pionków wzdłuż tych samych kolumn i ścieżek często zachowują swoją skuteczność bez względu na dokładną kolejność innych posunięć. Zaimplementowanie RAVE jest próbą wykorzystania tej właściwości do drastycznego poprawienia siły gry agenta przy niskich budżetach iteracyjnych.

2.3. Progressive Bias

Progressive Bias to technika z rodziny strategii progresywnych (opisanych m.in. przez Chaslot’a i in. [6]), służąca do wstrzykiwania z góry zdefiniowanej wiedzy domenowej (heurystyki) bezpośrednio do fazy selekcji w drzewie MCTS. Zamiast rozpoczynać przeszukiwanie „na ślepo”, algorytm na początku faworyzuje te gałęzie, które z perspektywy funkcji oceny wydają się najlepsze.

Formułę selekcji modyfikuje się poprzez dodanie elementu uwzględniającego wartość heurystyczną H_i dla danego ruchu i :

$$\text{Wartość} = UCT_i + \frac{H_i}{n_i + 1}$$

Kluczowym elementem tego równania jest mianownik. Początkowo parametr H_i ma ogromny wpływ na wybór ścieżki. W miarę gromadzenia statystyk i wzrostu wartości n_i (liczby odwiedzin węzła), rola heurystyki płynnie zanika na rzecz symulacji Monte Carlo. Pozwala to na połączenie mocnych stron obu podejść: silnego początku gwarantowanego przez wiedzę ludzką oraz teoretycznej zbieżności do optimum zapewnianej przez MCTS.

Wykorzystanie w projekcie: Zbudujemy specjalistyczną funkcję heurystyczną dla Breakthrough i włączymy ją w algorytm MCTS poprzez Progressive Bias. Pozwoli to zbadać, czy ukierunkowanie MCTS w początkowej fazie daje lepsze rezultaty niż poleganie na wariancie uczącym się od zera.

2.4. Heurystyka specjalistyczna dla Breakthrough

Przy tworzeniu heurystyki specjalistycznej wzięto pod uwagę następujące czynniki pozycyjne, z których każdy wnosi odrębną informację o jakości pozycji:

- **Zaawansowanie pionków:** Waga pionka wzrasta wykładniczo wraz ze zbliżaniem się do linii awansu — pionek w szóstym rzędzie jest wielokrotnie cenniejszy niż w drugim.
- **Groźba natychmiastowego przebicia:** Pionek w przedostatnim rzędzie, dysponujący co najmniej jednym legalnym ruchem na rząd awansu, generuje ogromną presję taktyczną.
- **Obrońcy (bezpieczeństwo):** Pionek wspierany przez sojuszników na przekątnych za nim jest praktycznie niemożliwy do zbitia bez poniesienia strat materialnych przez przeciwnika.
- **Centralność:** Pionki w środkowych kolumnach planszy kontrolują więcej pól i mają większą elastyczność ruchów niż pionki na krawędziach.
- **Swobodna ścieżka:** Pionek, przed którym (w trójkącie rozszerzającym się o jedną kolumnę na rząd) nie stoi żaden pionek przeciwnika, ma niezablokowaną drogę do awansu — analogia do pionka wolnego w szachach.
- **Izolacja:** Pionek pozbawiony jakichkolwiek sąsiadów ze swojej strony (na przekątnych za nim oraz na sąsiednich polach w tym samym rzędzie) jest podatny na bicie i trudny do obrony.
- **Mobilność:** Różnica w liczbie legalnych ruchów obu stron odzwierciedla presję pozycyjną i elastyczność w budowaniu ataków.
- **Przewaga materialna:** Różnica w liczbie pionków na planszy — czynnik drugorzędny w Breakthrough (pozycja jest ważniejsza niż materiał), ale istotny w końcówkach.

Wykorzystanie w projekcie: Powyższe elementy zostaną użyte do stworzenia funkcji oceny stanu gry. Posłuży ona do modyfikacji Progressive Bias w algorytmie MCTS, a także pozwoli stworzyć w pełni funkcjonalnego agenta bazującego na klasycznym przeszukiwaniu Minimax z odcięciami alfa-beta, który wystąpi jako rywal dla algorytmów opartych o Monte Carlo Tree Search.

Poniższe cztery algorytmy tworzą kompletnego agenta heurystycznego: funkcję ewaluacji pozycji (Algorytm 1), heurystykę porządkowania ruchów przyspieszającą odcięcia (Algorytm 2), przeszukiwanie minimax z odcięciami alfa-beta (Algorytm 3) oraz punkt wejścia wybierający najlepszy ruch (Algorytm 4).

Algorytm 1 Funkcja ewaluacji pozycji w Breakthrough**Dane:** Stan gry s (plansza 8×8 , zbiory pionków białych P_W i czarnych P_B)**Wynik:** Ocena liczbowa $v \in \mathbb{R}$; $v > 0$ faworyzuje białe, $v < 0$ faworyzuje czarne

```

1:  $W \leftarrow [1, 1, 2, 4, 8, 16, 32, 64]$   $\triangleright$  Tablica wag zaawansowania (indeks = rząd)
2: if  $s$  jest stanem terminalnym then return  $+\infty$  jeśli wygrywają białe,  $-\infty$  jeśli
   czarne
3:  $v_W \leftarrow 0$ 
    $\triangleright$  — Groźba natychmiastowego przebicia (białe, rząd awansu = 7) —
4: for każdy pionek  $p \in P_W$  na pozycji  $(r, c)$  gdzie  $r = 6$  do
5:   if pole  $(7, c)$  puste lub  $\exists c' \in \{c-1, c+1\}$ : pole  $(7, c')$  puste lub  $\in P_B$  then
6:      $v_W \leftarrow v_W + 200$   $\triangleright$  Groźba wygranej w jednym ruchu
        $\triangleright$  — Ocena pozycyjna każdego białego pionka —
7: for każdy pionek  $p \in P_W$  na pozycji  $(r, c)$  do
8:    $a \leftarrow W[r]$   $\triangleright$  Waga zaawansowania
9:    $d \leftarrow |\{(r-1, c-1), (r-1, c+1)\} \cap P_W|$   $\triangleright$  Liczba obrońców
10:   $k \leftarrow \begin{cases} 1,15 & \text{jeśli } c \in \{3, 4\} \\ 1,05 & \text{jeśli } c \in \{2, 5\} \\ 1,0 & \text{w p.p.} \end{cases}$   $\triangleright$  Bonus centralności
11:   $wolny \leftarrow \neg \exists p' \in P_B \text{ na } (r', c') \text{ t. że } r' > r \text{ i } |c' - c| \leq r' - r$   $\triangleright$  Swobodna
    ścieżka
12:   $f \leftarrow \begin{cases} 1,5 & \text{jeśli } wolny \\ 1,0 & \text{w p.p.} \end{cases}$ 
13:   $izol \leftarrow |\{(r-1, c-1), (r-1, c+1), (r, c-1), (r, c+1)\} \cap P_W| = 0$   $\triangleright$  Izolacja
14:   $g \leftarrow \begin{cases} 0,8 & \text{jeśli } izol \\ 1,0 & \text{w p.p.} \end{cases}$ 
15:   $v_W \leftarrow v_W + a \cdot (1 + 0,3 d) \cdot k \cdot f \cdot g$ 
    $\triangleright$  — Ocena czarnych ( $v_B$ ) — analogicznie z lustrzanym kierunkiem —
    $\triangleright$  — Czynniki globalne —
16:  $v \leftarrow v_W - v_B$ 
17:  $v \leftarrow v + 0,3 \cdot (|\text{LEGALNERUCHY}(s, W)| - |\text{LEGALNERUCHY}(s, B)|)$   $\triangleright$  Mobilność
18:  $v \leftarrow v + 2,0 \cdot (|P_W| - |P_B|)$   $\triangleright$  Przewaga materialna
19: return  $v$ 

```

Algorytm 2 Heurystyka porządkowania ruchów

Dane: Stan gry s , ruch $m = (skd, dokd)$ **Wynik:** Priorytet $\pi(m) \in \mathbb{R}$ (im wyższy, tym ruch sprawdzany wcześniej)

```
1:  $\pi \leftarrow 0$ 

2: if pole docelowe  $m.dokd$  leży na ostatnim rzędzie przeciwnika then
3:   return 1000                                ▷ Ruch wygrywający — najwyższy priorytet

4: if  $m$  jest biciem then
5:    $\pi \leftarrow \pi + 50$ 
6:    $\pi \leftarrow \pi + 5 \cdot \text{ZAAWANSOWANIE}(\text{zbity pionek})$                 ▷ Cenniejsze bicie
   zaawansowanego pionka

7: if pole  $m.dokd$  będzie bronione po wykonaniu ruchu then
8:    $\pi \leftarrow \pi + 10$                                 ▷ Preferuj bezpieczne pozycje

9:  $\pi \leftarrow \pi + 2 \cdot \text{ZAAWANSOWANIE}(m.dokd)$                 ▷ Preferuj ruchy do przodu

10: return  $\pi$ 
```

Algorytm 3 Minimax z odcięciami alfa-beta

Dane: Stan gry s , głębokość d , granice α , β , flaga $maks$ **Wynik:** Ocena najlepszej osiągalnej pozycji

```

1: if  $d = 0$  lub  $s$  jest stanem terminalnym then
2:   return EWALUACJA( $s$ )                                     ▷ Algorytm 1

3:  $R \leftarrow \text{LEGALNERUCHY}(s)$ 
4: Posortuj  $R$  malejąco wg  $\pi(m)$                                ▷ Algorytm 2

5: if  $maks = \text{prawda}$  then                                     ▷ Tura białych — maksymalizacja
6:    $v \leftarrow -\infty$ 
7:   for każdy  $m \in R$  do
8:      $s' \leftarrow \text{ZASTOSUJ}(s, m)$ 
9:      $v \leftarrow \max(v, \text{ALFABETA}(s', d-1, \alpha, \beta, \text{fałsz}))$ 
10:     $\alpha \leftarrow \max(\alpha, v)$ 
11:    if  $\alpha \geq \beta$  then
12:      przerwij                                               ▷ Odcięcie  $\beta$ 
13: else                                                         ▷ Tura czarnych — minimalizacja
14:    $v \leftarrow +\infty$ 
15:   for każdy  $m \in R$  do
16:      $s' \leftarrow \text{ZASTOSUJ}(s, m)$ 
17:      $v \leftarrow \min(v, \text{ALFABETA}(s', d-1, \alpha, \beta, \text{prawda}))$ 
18:      $\beta \leftarrow \min(\beta, v)$ 
19:     if  $\alpha \geq \beta$  then
20:       przerwij                                               ▷ Odcięcie  $\alpha$ 

21: return  $v$ 

```

Algorytm 4 Wybór najlepszego ruchu — punkt wejścia agenta heurystycznego**Dane:** Stan gry s , maksymalna głębokość przeszukiwania D **Wynik:** Najlepszy ruch m^*

```

1:  $R \leftarrow \text{LEGALNERUCHY}(s)$ 
2: Posortuj  $R$  malejąco wg  $\pi(m)$  ▷ Algorytm 2

3:  $maks \leftarrow (s.tura = \text{Białe})$ 
4:  $v^* \leftarrow \begin{cases} -\infty & \text{jeśli } maks \\ +\infty & \text{w p.p.} \end{cases}$ 
5: for każdy  $m \in R$  do
6:    $s' \leftarrow \text{ZASTOSUJ}(s, m)$ 
7:    $v \leftarrow \text{ALFABETA}(s', D-1, -\infty, +\infty, \neg maks)$ 
8:   if ( $maks$  i  $v > v^*$ ) lub ( $\neg maks$  i  $v < v^*$ ) then
9:      $v^* \leftarrow v$ 
10:     $m^* \leftarrow m$ 

11: return  $m^*$ 

```

3. Hipotezy badawcze**H1. RAVE poprawia skuteczność względem bazowego UCT.**

Agent UCT rozszerzony o RAVE osiąga istotnie wyższy win-rate niż czysty UCT przy tym samym budżecie iteracji.

H2. Progressive Bias poprawia skuteczność względem bazowego UCT.

Agent UCT z Progressive Bias osiąga istotnie wyższy win-rate niż czysty UCT, szczególnie przy niskich budżetach iteracji.

H3. Heurystyka specjalistyczna przewyższa MCTS przy małym budżecie iteracji, jednak przegrywa przy wysokim.

Agent minimax alfa-beta z heurystyką domenową wygrywa z UCT przy małych budżetach iteracji, ale przegrywa przy dużych.

H4. Przewaga pierwszego gracza.

W Breakthrough pierwszy gracz ma niewielką przewagę — podobna sytuacja jak w szachach.

H5. Stała eksploracji c w UCT ma nieliniowy wpływ na siłę gry.

Istnieje optymalna wartość c^* maksymalizująca win-rate agenta UCT w Breakthrough 8 × 8. Zarówno zbyt niska (nadmierna eksploatacja), jak i zbyt wysoka (nadmierna eksploracja) wartość c prowadzi do istotnego spadku skuteczności.

H6. Czas podejmowania decyzji MCTS rośnie w środkowej fazie gry.

Przy stałym budżecie iteracji średni czas na ruch agenta MCTS jest najwyższy w fazie środkowej partii (ruchy 15–40), gdy branching factor jest

największy, i maleje w otwarciu oraz końcówce.

4. Sposób weryfikacji hipotez

4.1. Projekt eksperymentów

Weryfikacja hipotez oparta jest na kontrolowanych turniejach pomiędzy agentami, przy zachowaniu identycznych budżetów obliczeniowych oraz deterministycznej inicjalizacji losowości.

Każda konfiguracja agentów rozgrywa serie partii w schemacie:

- **symetryczne role** — każdy agent gra równą liczbę partii białymi i czarnymi pionkami,
- **stały budżet obliczeniowy** — liczba iteracji MCTS lub głębokość przeszukiwania,
- **kontrola losowości** — jawne seedy zapewniające reprodukowalność.

Dla każdej hipotezy:

- **H1:** turniej RAVE vs UCT dla budżetów {5k, 10k, 50k, 100k} iteracji.
- **H2:** turniej Progressive Bias vs UCT dla tych samych budżetów.
- **H3:** turniej minimax (głębokość 5) vs UCT dla budżetów {500, 1k, 5k, 10k, 50k, 100k}.
- **H4:** symetryczne mecze UCT vs UCT.
- **H5:** przeszukiwanie siatki parametrów $c \in \{0.5, 1.0, 1.5, 2.0, 3.0\}$.
- **H6:** analiza czasu decyzji w funkcji numeru ruchu.

4.2. Dane eksperymentalne

Każda partia generuje rekord w formacie JSONL zawierający:

- konfigurację agentów (algorytm, hiperparametry),
- seed losowy,
- sekwencję ruchów,
- wynik (wygrana/przegrana),
- długość partii,
- czas decyzji dla każdego ruchu.

Dane są zapisywane w sposób append-only, co umożliwia wznowienie eksperymentów bez utraty wyników.

4.3. Liczność prób

Dla każdej konfiguracji rozegranych zostanie:

- **100 partii** (50 na każdą stronę),

4.4. Metodologia statystyczna

Dla każdej konfiguracji obliczane są:

- **win rate** wraz z 95% przedziałem ufności Wilsona,
- **test istotności** (test dwumianowy lub test proporcji) względem hipotezy $p = 0.5$,
- **effect size** (różnica win rate względem baseline).

4.5. Metryki dodatkowe

Poza win rate analizowane są:

- **długość partii** (mediana, IQR),
- **czas na ruch** (średnia i rozkład),
- **stabilność decyzji** (wariancja wyników dla tych samych seedów),
- **efektywność obliczeniowa** (win rate na jednostkę czasu).

4.6. Analiza wyników

Wyniki prezentowane są w formie:

- krzywych win rate vs budżet iteracji,
- heatmap dla parametrów (np. c w UCT),
- boxplotów długości partii,
- wykresów czasu decyzji w funkcji numeru ruchu.

4.7. Reprodukowalność

Eksperymenty są w pełni deterministyczne:

- każdy eksperyment posiada master seed,
- seed każdej partii wyznaczany jest jako funkcja (master, game_id),
- wszystkie konfiguracje zapisywane są wraz z wynikami.

4.8. Testy z udziałem użytkowników

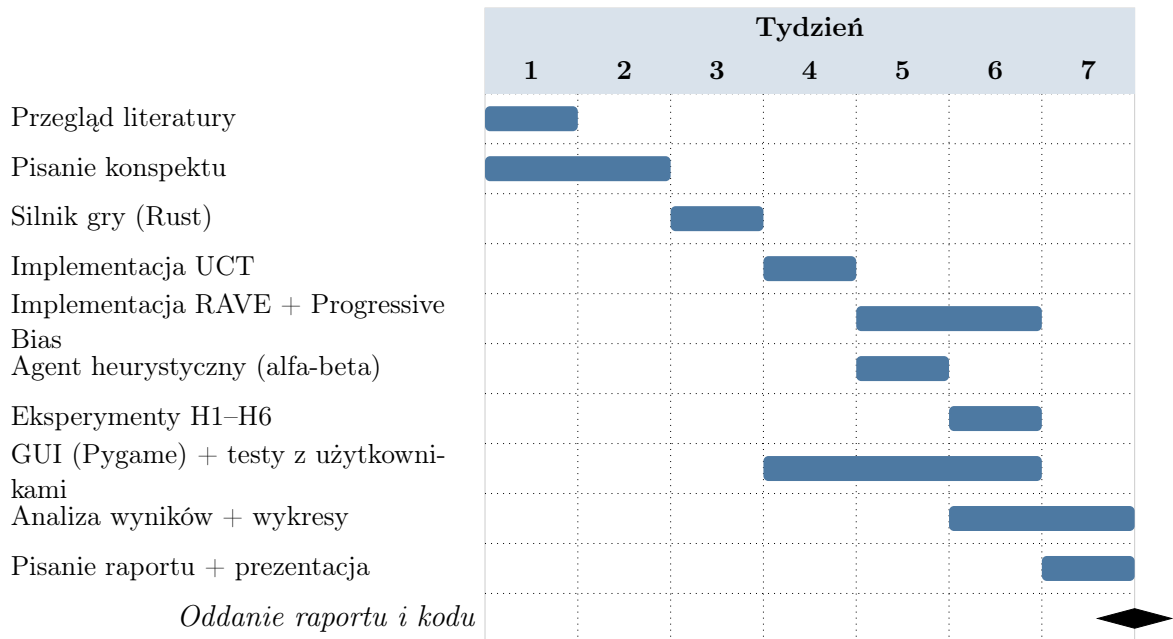
Interfejs graficzny testowany jest przez co najmniej 5 użytkowników.

Zbierane dane:

- wyniki partii człowiek-agent,
- długość partii,
- subiektywna ocena poziomu trudności (skala 1 – 10).

Celem jest jakościowa ocena siły agentów oraz wykrycie nieludzkich lub nieintuicyjnych zachowań algorytmu.

5. Harmonogram projektu



Rysunek 1: Harmonogram projektu — diagram Gantta

6. Ogólny projekt techniczny

6.1. Stos technologiczny

Rdzeń obliczeniowy — silnik gry, wszystkie warianty MCTS, heurystyka — implementujemy w języku **Rust**. Orchestracja eksperymentów, interfejs graficzny i analiza wyników — w języku **Python 3.12**. Wiązanie Rust↔Python poprzez PyO3 z narzędziem maturin (build). Wybór Rusta motywowany wydajnością (MCTS z dużym budżetem iteracji dominuje w czasie obliczeń) i bezpieczeństwem pamięci.

6.2. Zrównoleglenie

Kluczowa decyzja metodologiczna: **MCTS działa sekwencyjnie, zrównoleglenie jest wyłącznie na poziomie batchowania eksperymentów**. Każda partia turnieju to osobny proces Python (ProcessPoolExecutor). Nie wprowadzamy tree parallelization ani root parallelization. Uzasadnienie: czystość metodologiczna porównań — porównujemy warianty algorytmiczne, nie wydajność zrównoleglenia. Budżet MCTS mierzony liczbą iteracji pozostaje jednoznaczny.

6.3. Struktura repozytorium

```
breakthrough-mcts/
  Cargo.toml, pyproject.toml    # build
  src/                          # Rust
    game.rs                     # bitboardy, ruchy, terminale
    mcts/{uct,rave,progressive_bias}.rs
    heuristic.rs                 # alfa-beta
```

python/breakthrough/	# Python
gui.py	# Pygame
experiments/	
notebooks/	# skrypty analityczne
results/	# JSONL

Bibliografia

- [1] Abdallah Saffidine, Nicolas Jouandeau i Tristan Cazenave. “Solving Breakthrough with Race Patterns and Job-Level Proof Number Search”. W: *Advances in Computer Games (ACG 2011)*. T. 7168. Lecture Notes in Computer Science. Springer, 2012, s. 196–207. DOI: [10.1007/978-3-642-31866-5_17](https://doi.org/10.1007/978-3-642-31866-5_17).
- [2] Yngvi Björnsson i in. “Solving Breakthrough for the 6×6 Board”. W: *ICGA Journal* (2026). Online first, March 2026. DOI: [10.1177/13896911261430592](https://doi.org/10.1177/13896911261430592).
- [3] Rémi Coulom. “Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search”. W: *Computers and Games (CG 2006)*. T. 4630. Lecture Notes in Computer Science. Springer, 2006, s. 72–83. DOI: [10.1007/978-3-540-75538-8_7](https://doi.org/10.1007/978-3-540-75538-8_7).
- [4] Levente Kocsis i Csaba Szepesvári. “Bandit Based Monte-Carlo Planning”. W: *Machine Learning: ECML 2006*. T. 4212. Lecture Notes in Computer Science. Springer, 2006, s. 282–293. DOI: [10.1007/11871842_29](https://doi.org/10.1007/11871842_29).
- [5] Sylvain Gelly i David Silver. “Monte-Carlo Tree Search and Rapid Action Value Estimation in Computer Go”. W: *Artificial Intelligence* 175.11 (2011), s. 1856–1875. DOI: [10.1016/j.artint.2011.03.007](https://doi.org/10.1016/j.artint.2011.03.007).
- [6] Guillaume Chaslot i in. “Progressive Strategies for Monte-Carlo Tree Search”. W: *New Mathematics and Natural Computation* 4.3 (2008), s. 343–357. DOI: [10.1142/S1793005708001094](https://doi.org/10.1142/S1793005708001094).